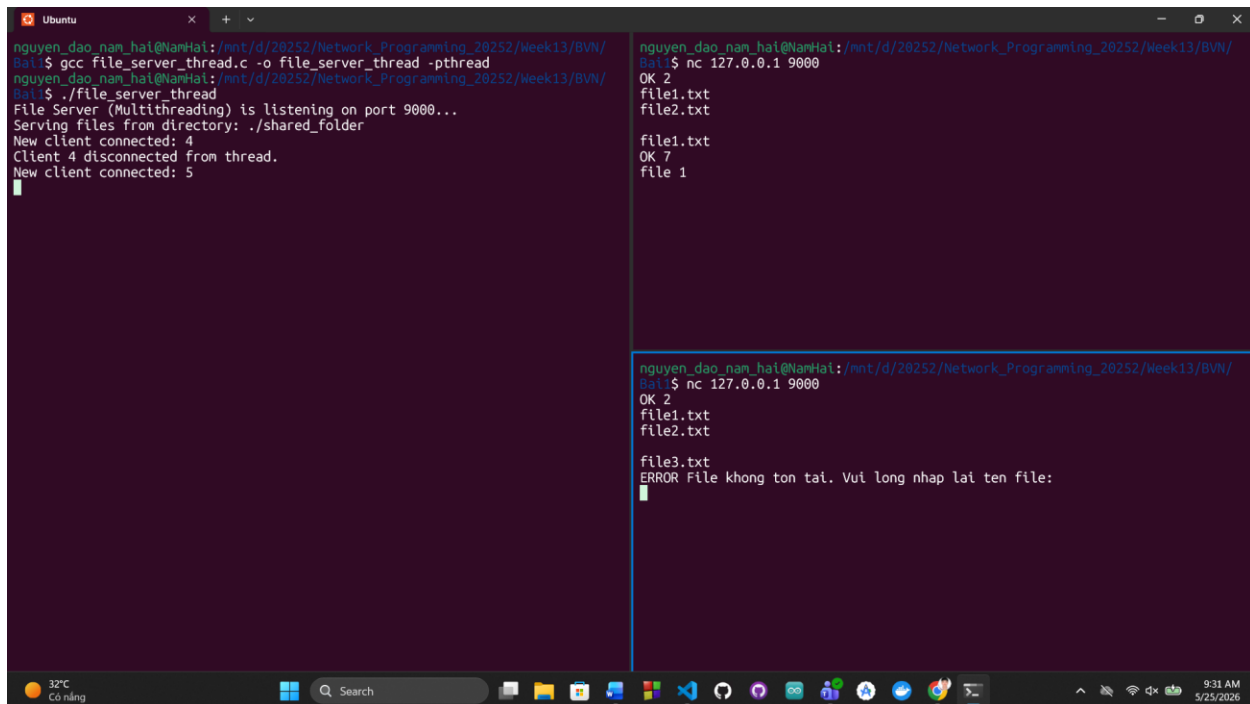


Bài tập 03.01

Sử dụng kỹ thuật đa tiến trình, lập trình ứng dụng **file server** đơn giản như sau:

- Khi client mới được chấp nhận, server sẽ gửi danh sách các file cho client. Các file này trong thư mục được thiết lập trên server. Dòng đầu danh sách là chuỗi “**OK N\r\n**” nếu có N file trong thư mục. Các tên file phân cách nhau bởi ký tự `\r\n`, kết thúc danh sách bởi ký tự `\r\n\r\n`.
- Nếu không có file nào trong thư mục thì server gửi chuỗi “**ERROR No files to download\r\n**” rồi đóng kết nối.
- Client gửi tên file để tải về, server kiểm tra nếu file tồn tại thì gửi chuỗi “**OK N\r\n**” trong đó N là kích thước file, sau đó là nội dung file cho client và đóng kết nối, nếu file không tồn tại thì gửi thông báo lỗi và yêu cầu client gửi lại tên file.

Demo:



```
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: gcc file_server_thread.c -o file_server_thread -pthread
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: ./file_server_thread
File Server (Multithreading) is listening on port 9000...
Serving files from directory: ./shared_folder
New client connected: 4
Client 4 disconnected from thread.
New client connected: 5

nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: nc 127.0.0.1 9000
OK 2
file1.txt
file2.txt

file1.txt
OK 7
file 1

nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: nc 127.0.0.1 9000
OK 2
file1.txt
file2.txt

file3.txt
ERROR File không tồn tại. Vui lòng nhập lại tên file:
```

Giải thích:

Phía Server (Kiểm chứng luồng xử lý độc lập):

Terminal trái cho thấy Server dùng pthread để cấp cho mỗi Client một luồng (thread) riêng biệt. Các kết nối mới (FD 4, FD 5) được tiếp nhận và phục vụ liên tục. Đáng chú ý, log Client 4 disconnected from thread. chứng minh hệ thống đã xử lý xong yêu cầu, tự động đóng kết nối và dọn dẹp bộ nhớ của luồng đó sạch sẽ.

Phía Client (Kiểm chứng Đa nhiệm & Bắt lỗi):

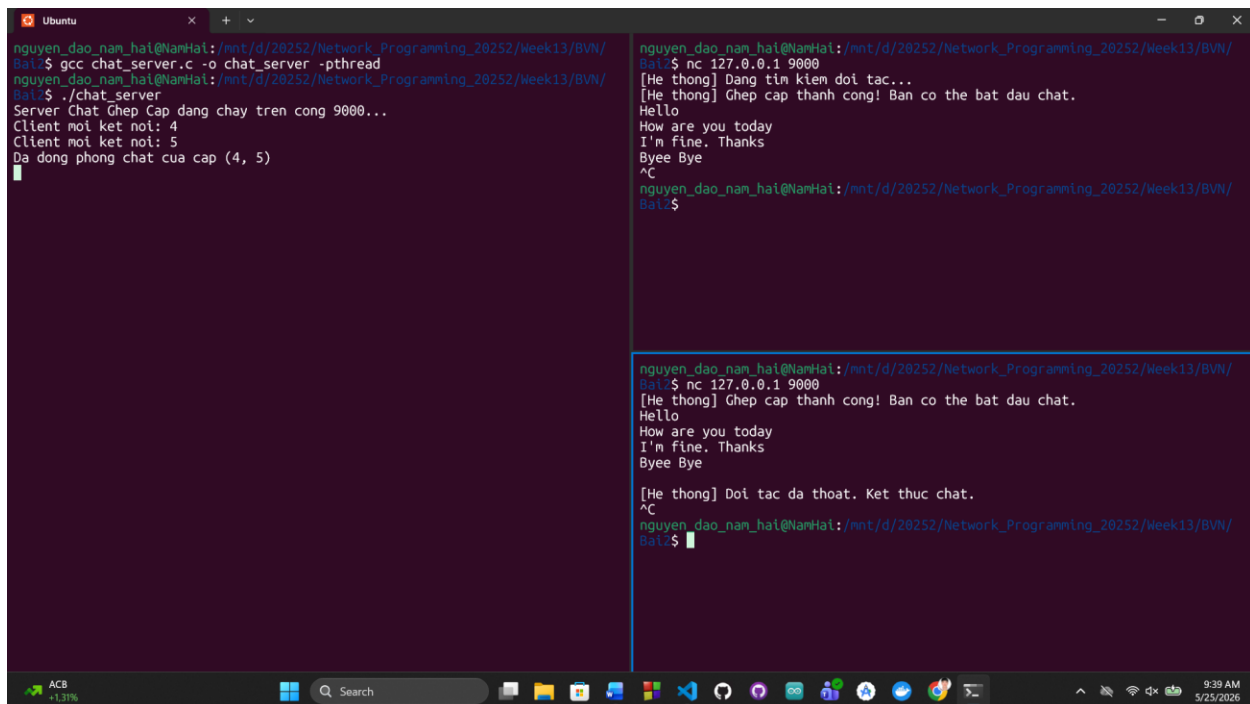
- **Luồng tải file thành công (Góc trên phải):** Client 1 (FD 4) yêu cầu file1.txt. Server tính toán chính xác kích thước file (OK 7), trả về đúng nội dung và ngắt kết nối theo đúng yêu cầu đề bài.
- **Luồng xử lý ngoại lệ (Góc dưới phải):** Client 2 (FD 5) kết nối song song và cố tình nhập sai tên (file3.txt). Luồng phụ trách Client 2 lập tức bắt lỗi *"ERROR File không tồn tại..."* và giữ phiên kết nối chờ người dùng nhập lại. Quá trình này chạy hoàn toàn độc lập, không làm ảnh hưởng đến luồng của các Client khác.

Bài tập 03.02

Sử dụng kỹ thuật đa luồng, lập trình ứng dụng chat nhóm 2 người như sau:

- Mỗi client khi kết nối đến server sẽ được lưu vào hàng đợi, nếu số lượng client trong hàng đợi là 2 thì 2 client này sẽ được ghép cặp với nhau.
- Khi 2 client đã được ghép cặp thì tin nhắn gửi từ client này sẽ được chuyển tiếp sang client kia và ngược lại.
- Nếu 1 trong 2 client ngắt kết nối thì client còn lại cũng được ngắt kết nối.

Demo:



```
nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bat:$ gcc chat_server.c -o chat_server -pthread
nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bat:$ ./chat_server
Server Chat Ghep Cap đang chạy trên cổng 9000...
Client mới kết nối: 4
Client mới kết nối: 5
Đã đóng phòng chat của cặp (4, 5)

nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bat:$ nc 127.0.0.1 9000
[He thong] Đang tìm kiếm đối tác...
[He thong] Ghep cap thanh cong! Ban co the bat dau chat.
Hello
How are you today
I'm fine. Thanks
Byee Bye
^C
nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bat:$

nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bat:$ nc 127.0.0.1 9000
[He thong] Ghep cap thanh cong! Ban co the bat dau chat.
Hello
How are you today
I'm fine. Thanks
Byee Bye

[He thong] Doi tac da thoat. Ket thuc chat.
^C
nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bat:$
```

Giải thích:

Cơ chế Hàng đợi & Ghép cặp (Waiting & Pairing)

- **Client 1 (Góc trên phải):** Kết nối đầu tiên (được cấp File Descriptor FD 4), hệ thống đưa vào hàng đợi và phát thông báo chờ (Đang tìm kiếm đối tác...).
- **Client 2 (Góc dưới phải):** Kết nối ngay sau đó (FD 5). Khóa mutex trên Server hoạt động chuẩn xác, lập tức lấy Client 1 ra khỏi hàng đợi để ghép cặp với Client 2 và mở phòng chat.

Truyền tin Song công (Message Routing) Quá trình trò chuyện diễn ra mượt mà (chào hỏi "Hello", "How are you today"...). Luồng (Thread) quản lý phòng chat này sử dụng hàm poll() để lắng nghe liên tục hai FD 4 và 5, giúp tin nhắn được luân chuyển hai chiều ngay lập tức mà không bị nghẽn (blocking) chờ nhập liệu.

Xử lý Ngắt kết nối đồng bộ (Disconnection Handling) Đây là tính năng quan trọng nhất đáp ứng đúng yêu cầu khắt khe của đề bài:

- Khi Client 1 đột ngột ngắt kết nối bằng tổ hợp phím Ctrl + C (^C), hàm recv() trên Server lập tức phát hiện kết nối bị đứt.
- Server tự động gửi thông báo *"Doi tac da thoat. Ket thuc chat."* sang Client 2, sau đó ép đóng kênh của Client 2.
- Phía Server in ra log Da dong phong chat cua cap (4, 5) chứng tỏ Thread đã tự động dọn dẹp sạch sẽ bộ nhớ và socket, không để lại rác hệ thống (Zombie Thread).

- Lập trình ứng dụng **chat_server** ([slide 173](#)) sử dụng kỹ thuật multithread.

Bài tập – Chat Server

Sử dụng hàm `select()/poll()`, viết chương trình **chat_server** thực hiện các chức năng sau:

Nhận kết nối từ các client, và vào hỏi tên client cho đến khi client gửi đúng cú pháp:

“client_id: client_name”

trong đó `client_name` là tên của client, xâu ký tự viết liền.

Sau đó nhận dữ liệu từ một client và gửi dữ liệu đó đến các client còn lại, ví dụ: client có id “abc” gửi “xin chào” thì các client khác sẽ nhận được: “abc: xin chào” hoặc có thể thêm thời gian vào trước ví dụ: “2023/05/06 11:00:00PM abc: xin chào”.

Demo:

The screenshot shows two terminal windows on an Ubuntu system. The left window shows the server's execution: compilation of chat_server.c, running the server on port 8080, and receiving connections from Alice and Bob. The right window shows the interaction from two clients. Client 1 (top) successfully registers as Alice and sends messages. Client 2 (bottom) initially enters an incorrect command 'Bob Bob', receives an error, and then successfully registers as Bob and sends messages. The messages are timestamped and include the sender's ID.

```
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: $ gcc chat_server.c -o chat_server
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: $ ./chat_server
Chat Room Server (Multithreading) đang chạy trên cổng 8080...
Client Alice đã tham gia phòng chat.
Client Bob đã tham gia phòng chat.

nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: $ nc 127.0.0.1 8080
Vui lòng nhập theo cú pháp: client_id: <ten_viet_lien>
client_id: Alice
Đang nhập thành công! Bạn có thể bắt đầu chat.
2026/05/25 02:45:32AM Bob: Hello, my name is Bob
Hi, I'm Alice
Nice to meet you
see you later
2026/05/25 02:45:51AM Bob: okay bye

nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: $ nc 127.0.0.1 8080
Vui lòng nhập theo cú pháp: client_id: <ten_viet_lien>
Bob Bob
Sai cú pháp! Vui lòng nhập lại (VD: client_id: NamHai):
client_id: Bob
Đang nhập thành công! Bạn có thể bắt đầu chat.
Hello, my name is Bob
2026/05/25 02:45:40AM Alice: Hi, I'm Alice
2026/05/25 02:45:45AM Alice: Nice to meet you
2026/05/25 02:45:48AM Alice: see you later
okay bye
```

Giải thích:

Kiểm chứng cơ chế Đăng nhập & Bắt lỗi (Authentication)

- Tại Terminal góc dưới phải (Client 2), người dùng cố tình nhập sai cú pháp khai báo ban đầu (Bob Bob). Luồng (Thread) quản lý Client này đã bắt lỗi thành công, từ chối quyền truy cập và yêu cầu nhập lại.
- Khi cả 2 Client nhập chuẩn cú pháp client_id: <tên>, Server (Terminal trái) ngay lập tức ghi nhận danh tính (Alice và Bob) và lưu vào mảng quản lý chung.

Kiểm chứng luồng Broadcast & Thời gian thực (Timestamp)

- Quá trình nhắn tin diễn ra mượt mà theo thời gian thực. Khi Bob gửi câu "Hello, my name is Bob", hàm broadcast_message trên Server đã duyệt qua mảng danh sách người dùng và gửi tin nhắn đó đến Alice (góc trên phải).
- Đặc biệt, hệ thống đã xử lý chuỗi thành công khi tự động gắn thêm **Thời gian thực (Timestamp)** và **Tên người gửi (ID)** vào trước nội dung tin nhắn (Ví dụ: 2026/05/25 02:45:40AM Alice: Hi, I'm Alice).

- Lập trình ứng dụng **telnet_server** ([slide 174](#)) sử dụng kỹ thuật multithread.

Bài tập – Telnet Server

Sử dụng hàm `select()/poll()`, viết chương trình **telnet_server** thực hiện các chức năng sau:

Khi đã kết nối với 1 client nào đó, yêu cầu client gửi user và pass, so sánh với file cơ sở dữ liệu là một file text, mỗi dòng chứa một cặp user + pass ví dụ:

admin admin

guest nopass

...

Nếu so sánh sai, không tìm thấy tài khoản thì báo lỗi đăng nhập.

Nếu đúng thì đợi lệnh từ client, thực hiện lệnh và trả kết quả cho client.

Dùng hàm `system("dir > out.txt")` để thực hiện lệnh.

dir là ví dụ lệnh dir mà client gửi.

> out.txt để định hướng lại dữ liệu ra từ lệnh dir, khi đó kết quả lệnh dir sẽ được ghi vào file văn bản.

Demo:

```
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: gcc telnet_server.c -o telnet_server
nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: ./telnet_server
Telnet Server (Multithreading) đang chạy trên cổng 8080...
Client mới kết nối: 4
Client mới kết nối: 5

nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: nc 127.0.0.1 8080
Vui lòng nhập 'user pass' để đăng nhập:
admin 123
Sai tài khoản. Hãy nhập lại:
admin admin
Đăng nhập thành công. Hãy nhập lệnh:
ls -la
total 24
drwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai 4096 May 25 02:52 .
drwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai 4096 May 25 02:48 ..
-rwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai 0 May 25 02:52 out_fd_4.txt
-rwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai 17104 May 25 02:51 telnet_server
-rwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai 3748 May 25 02:48 telnet_server.c
-rwxrwxrwx 1 nguyen_dao_nam_hai nguyen_dao_nam_hai 38 May 25 02:51 users.txt

nguyen_dao_nam_hai@NamHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal: nc 127.0.0.1 8080
Vui lòng nhập 'user pass' để đăng nhập:
guest nopass
Đăng nhập thành công. Hãy nhập lệnh:
cat users.txt
admin admin
guest nopass
guest nopass
```

Giải thích:

Cơ chế Đa luồng (Server - Góc trái): Server ghi nhận 2 kết nối đến từ Client (File Descriptor là 4 và 5) và đã sử dụng pthread để tạo ra 2 luồng (thread) phục vụ hoàn toàn độc lập, chạy song song cùng lúc.

Cơ chế Xác thực (Client 1 - Góc trên phải): Quá trình đăng nhập hoạt động chặt chẽ. Khi Client 1 nhập sai mật khẩu (admin 123), hệ thống từ chối và yêu cầu nhập lại. Chỉ khi nhập đúng (admin admin), Server mới cấp quyền thực thi lệnh.

Thực thi lệnh & Chống xung đột đa luồng (Điểm nhấn):

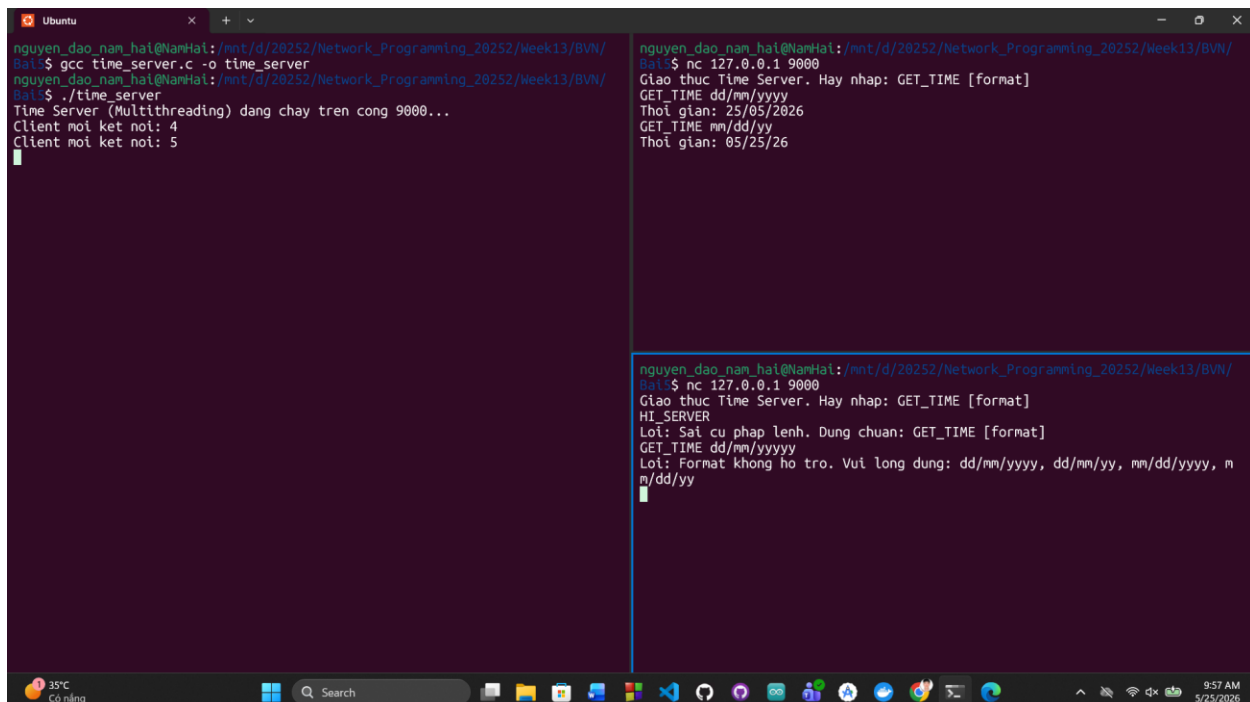
- **Tại Client 1:** Gõ lệnh `ls -la`. Điểm "ăn tiền" nhất trong ảnh là kết quả trả về xuất hiện file **out_fd_4.txt**. Điều này chứng minh thuật toán dùng mã File Descriptor (FD=4) để đặt tên file tạm đã hoạt động hoàn hảo. Nó giúp luồng của Client 1 lưu kết quả độc lập mà không sợ bị luồng khác ghi đè lên.
- **Tại Client 2 (Góc dưới phải):** Đăng nhập bằng `guest nopass` và gọi lệnh `cat users.txt` thành công. Tiến trình đọc file này diễn ra trơn tru, lấy được dữ liệu trả về mà không hề xảy ra xung đột bộ nhớ (Race Condition) với quá trình ghi file của Client 1.

- Lập trình ứng dụng **time_server** ([slide 204](#)) sử dụng kỹ thuật multithread.

Bài tập

- Lập trình ứng dụng **time_server** thực hiện chức năng sau:
 - Chấp nhận nhiều kết nối từ các client sử dụng kỹ thuật multiprocessing.
 - Client gửi lệnh GET_TIME [format] để nhận thời gian từ server.
 - **format** là định dạng thời gian server cần trả về client. Các format cần hỗ trợ gồm:
 - dd/mm/yyyy – vd: 30/01/2023
 - dd/mm/yy – vd: 30/01/23
 - mm/dd/yyyy – vd: 01/30/2023
 - mm/dd/yy – vd: 01/30/23
 - Cần kiểm tra tính đúng đắn của lệnh client gửi lên server.

Demo:



```
nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal$ gcc time_server.c -o time_server
nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal$ ./time_server
Time Server (Multithreading) đang chạy trên cổng 9000...
Client mới kết nối: 4
Client mới kết nối: 5

nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal$ nc 127.0.0.1 9000
Giao thức Time Server. Hay nhập: GET_TIME [format]
GET_TIME dd/mm/yyyy
Thời gian: 25/05/2026
GET_TIME mm/dd/yy
Thời gian: 05/25/26

nguyen_dao_nam_hai@NanHai: /mnt/d/20252/Network_Programming_20252/Week13/BVN/
Bal$ nc 127.0.0.1 9000
Giao thức Time Server. Hay nhập: GET_TIME [format]
HI_SERVER
Lỗi: Sai cú pháp lệnh. Dùng chuẩn: GET_TIME [format]
GET_TIME dd/mm/yyyyyy
Lỗi: Format không hỗ trợ. Vui lòng dùng: dd/mm/yyyy, dd/mm/yy, mm/dd/yyyy, m
m/dd/yy
```

Giải thích:

Khả năng phục vụ Đa luồng (Góc trái): Server đã tiếp nhận thành công 2 kết nối cùng lúc (cấp File Descriptor 4 và 5). Nhờ sử dụng pthread, hệ thống tạo ra 2 luồng (thread) riêng biệt để phục vụ 2 Client này chạy song song mà không bị hiện tượng "người sau phải chờ người trước".

Xử lý lệnh hợp lệ & Định dạng thời gian (Client 1 - Góc trên phải): Client gửi đúng chuẩn lệnh GET_TIME kèm các format hợp lệ. Luồng xử lý đã bóc tách chuỗi thành công, lấy thời gian thực của hệ thống và dùng hàm strftime để ép kiểu trả về chính xác (Ví dụ: dd/mm/yyyy trả về 25/05/2026).

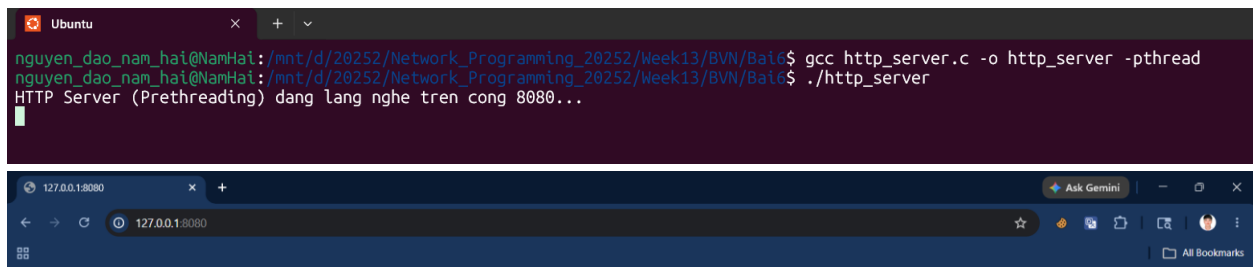
Cơ chế Bắt lỗi & Độc lập luồng (Client 2 - Góc dưới phải): Client 2 cố tình thử thách hệ thống bằng cách gửi sai cú pháp (HI_SERVER) và sai định dạng (dd/mm/yyyyy). Ngay lập tức, Server từ chối và trả về thông báo hướng dẫn sử dụng. Điểm sáng nhất là quá trình báo lỗi này chỉ diễn ra trong luồng của Client 2, hoàn toàn **không làm gián đoạn** quá trình xem giờ của Client 1.

- Lập trình ứng dụng **http_server** ([slide 154](#)) sử dụng kỹ thuật prethreading.

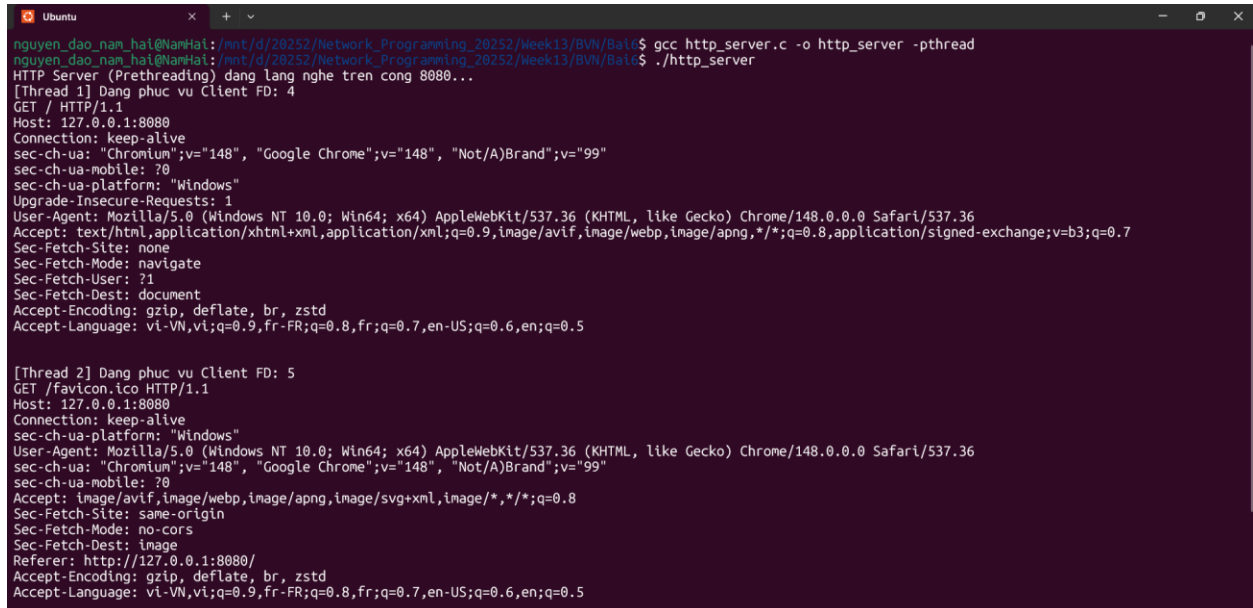
VD: HTTP server

```
// Khởi tạo server
while (1) {
    // Chờ kết nối mới
    int client = accept(listener, NULL, NULL);
    printf("New client connected: %d\n", client);
    // Nhận dữ liệu từ client và in ra màn hình
    char buf[256];
    int ret = recv(client, buf, sizeof(buf), 0);
    buf[ret] = 0;
    puts(buf);
    // Trả lại kết quả cho client
    char *msg = "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n<html><body><h1>Xin  
chao cac ban</h1></body></html>";
    send(client, msg, strlen(msg), 0);
    // Đóng kết nối
    close(client);
}
```

Demo:



Xin chao cac ban



Giải thích:

Khởi tạo Server & Thread Pool:

Chương trình được biên dịch chuẩn xác với thư viện đa luồng. Khi chạy lệnh khởi động, tiến trình chính (Main Thread) đã khởi tạo sẵn một đội ngũ luồng (Thread Pool) và bắt đầu lắng nghe trên cổng 8080. Lúc này, các luồng công nhân (Worker Threads) đang ở trạng thái ngủ đông (pthread_cond_wait) để không làm tiêu hao tài nguyên CPU của hệ thống.

Xử lý Giao thức HTTP:

Trình duyệt web (đóng vai trò là Client) thực hiện kết nối tới địa chỉ 127.0.0.1:8080. Toàn bộ quá trình ngầm diễn ra cực kỳ nhanh chóng:

- Luồng chính của Server bắt được kết nối, đẩy vào Hàng đợi (Queue) và đánh thức 1 luồng công nhân rảnh rồi đẩy làm việc.
- Luồng công nhân nhận gói tin yêu cầu (HTTP Request) từ trình duyệt, sau đó đóng gói trả về mã trạng thái HTTP/1.1 200 OK kèm theo nội dung văn bản HTML.
- **Kết quả hiển thị:** Trình duyệt nhận được gói tin, phân tích (parse) thành công thẻ <h1> và render ra dòng chữ "**Xin chào các bạn**" ngay trên giao diện đồ họa.

Server đã bắt được **2 yêu cầu (HTTP Requests)** liên tiếp từ trình duyệt và phân bổ chúng cực kỳ thông minh:

- **Yêu cầu 1 (Giao diện chính):** Khi truy cập `http://127.0.0.1:8080`, trình duyệt gửi yêu cầu GET / HTTP/1.1. Khách hàng này (FD: 4) được đưa vào hàng đợi và **[Thread 1]** đã thức dậy tiếp nhận để trả về đoạn mã HTML "Xin chào các bạn".
- **Yêu cầu 2 (Tài nguyên ngầm ngẫu nhiên):** Điểm thú vị của các trình duyệt hiện đại (Chrome/Edge) là ngay sau khi tải xong HTML trang chủ, nó sẽ tự động "bắn" thêm một yêu cầu ngầm thứ hai: GET /favicon.ico HTTP/1.1 để tìm cái biểu tượng (icon) nhỏ hiển thị trên tab trình duyệt. Vì đây là 2 yêu cầu gần như song song, hệ thống lập tức phân công **[Thread 2]** ra tiếp nhận khách hàng mới này (FD: 5).